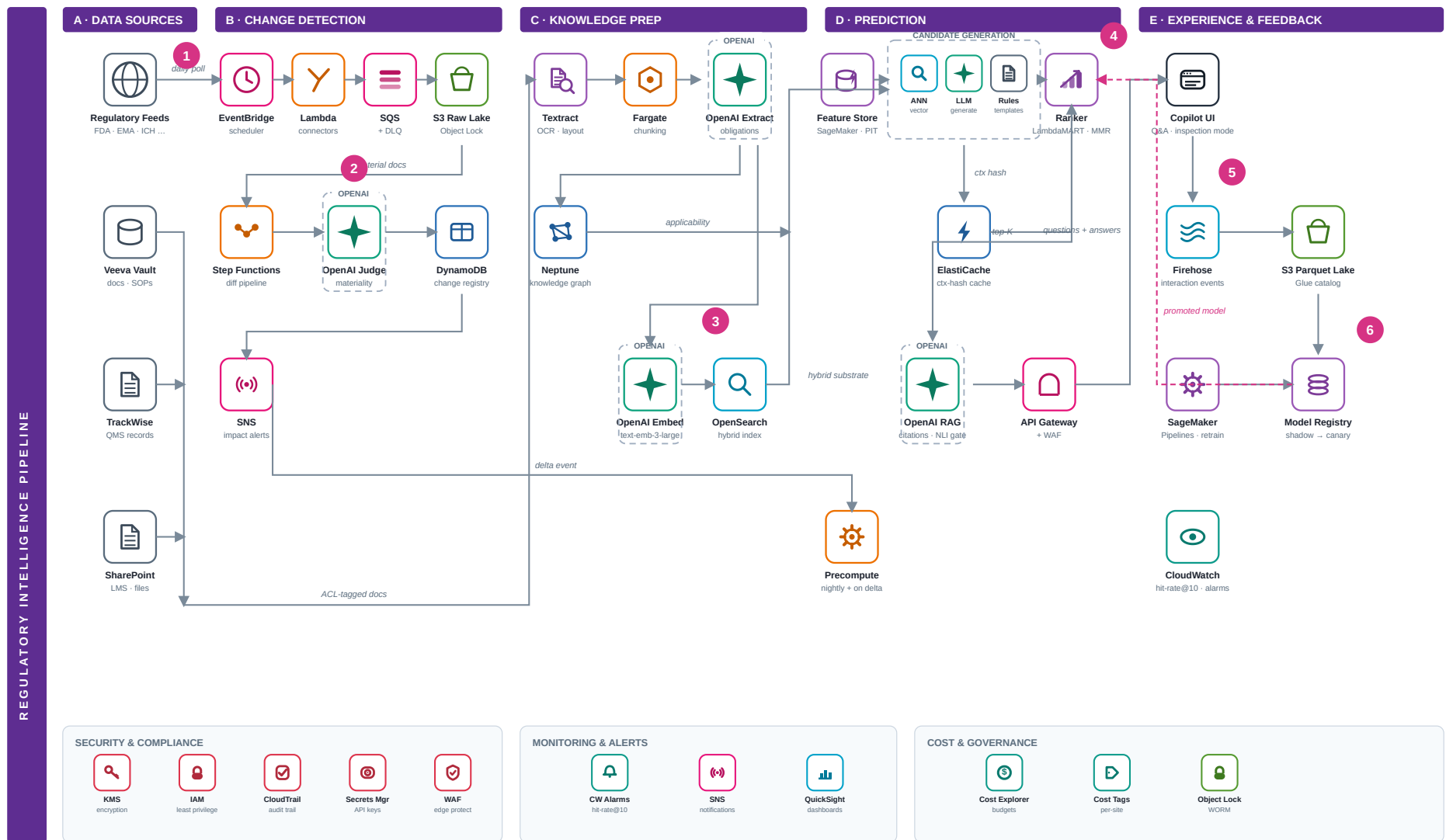


Regulatory AI question prediction

Architecture rationale

Senior/staff ML system-design interview companion
Box-by-box technology decisions, alternatives, limits, and traps

REGULATORY AI — QUESTION PREDICTION ON AWS



Six boundaries make the design defensible

The architecture is intentionally split where durability, latency, trust, or ownership changes. In an interview, state what crosses each boundary, what delivery guarantee applies, and how the next stage can replay or reject it.

1

Ingest

Source bytes + provenance

Regulatory and internal connectors write raw bytes, source IDs, ETags, retrieval timestamps, ACLs, and content hashes. The boundary ends only after S3 durably accepts the version and the manifest is idempotently registered.

Contract: at-least-once delivery; duplicate-safe document-version key; per-source freshness SLO.

2

Detect change

Aligned evidence + materiality decision

The diff workflow reads old/new immutable versions, normalizes structure, aligns changed spans, and asks the semantic judge only about bounded differences. It writes a structured decision and routes uncertainty to review.

Contract: source-span grounding; prompt/model version; false-negative budget; replay from S3.

3

Extract & index

Typed obligations + retrieval projections

Layout-aware text becomes stable chunks and structured obligations. Graph edges encode applicability and provenance; lexical and vector projections provide fast discovery. Both reference the same canonical version IDs.

Contract: exact spans; ACL metadata; blue/green index version; graph/search reconciliation.

4

Rank & serve

Context + candidates -> ordered questions

Point-in-time features combine with retrieval, generation, and template candidates. LambdaMART scores the candidates; MMR diversifies the list; cached results return before asynchronous RAG answers.

Contract: context fingerprint; score/model versions; p99 list latency; permission scope.

5

Capture feedback

Exposure-aware interaction events

The UI records what was shown, at which rank, what the analyst opened, what was actually asked, and what was corrected. Firehose lands immutable, duplicate-tolerant events in the Parquet lake.

Contract: event ID; prediction/session ID; exposure position; event and ingestion time.

6

Retrain & promote

Reproducible model package + rollout evidence

Pipelines rebuild labels with point-in-time joins, evaluate temporal and site slices, and register only passing models. Shadow and sticky canary stages generate evidence before production promotion.

Contract: dataset manifest; feature/code/model lineage; approval gates; atomic rollback set.

Regulatory feeds — FDA, EMA, ICH

Authoritative, provenance-rich ingestion for heterogeneous sources that rarely offer event streams.

1. ROLE

Provides the authoritative external stream of regulations, guidance, enforcement notices, warning letters, and inspection signals. Every item enters with source, jurisdiction, publication time, retrieval time, and immutable content hash.

2. TECHNOLOGY RATIONALE

The primitive is a heterogeneous pull-ingestion boundary: HTTP APIs, RSS/Atom feeds, sitemaps, and controlled crawlers normalized into one envelope. Polling is appropriate because most regulators do not publish transactional event streams; conditional requests with ETag and If-Modified-Since reduce bandwidth, while per-source watermarks make replay deterministic. The workload is low-to-moderate throughput but high consequence, so freshness and provenance matter more than raw QPS.

3. WHY THIS OVER ALTERNATIVES

Commercial regulatory-data aggregators reduce connector work but obscure source lineage, revision timing, and coverage gaps; they may remain a secondary feed, not the system of record. **Direct scraping into the index** is faster to prototype but couples source failures to serving and loses the raw evidence needed to replay parsers. **Webhook-only ingestion** is ruled out because most authorities do not offer dependable callbacks.

4. DISCUSSION POINTS

- Treat 404, 429, layout change, and silent feed staleness as distinct failure classes; “no new documents” is not proof the connector is healthy.
- Expect revisions without URL changes, duplicate publication across portals, and retroactively changed effective dates; hash normalized content, not just URLs.
- The interview trap is promising real-time regulatory awareness. Define a source-specific freshness SLO and a missing-source alarm instead.

Veeva Vault — documents and SOPs

Preserves regulated document lifecycle, versions, and ACLs from the system of record.

1. ROLE

Ingests controlled internal procedures, policies, effective versions, approvals, and document relationships used to answer predicted questions. The connector preserves Vault document IDs, lifecycle state, version, and access-control metadata.

2. TECHNOLOGY RATIONALE

The primitive is an API-backed regulated document repository with lifecycle semantics, not merely file storage. Incremental extraction should follow supported APIs and audit/event exports, writing a checkpoint only after the downstream durable landing succeeds. Throughput is usually batch-oriented; consistency is source-controlled, and downstream search should expose only effective versions unless an auditor explicitly requests history.

3. WHY THIS OVER ALTERNATIVES

Direct database access is unsupported, bypasses application authorization, and creates an unstable schema dependency. **Periodic CSV exports** are simple but lose document relationships, fine-grained ACLs, and timely lifecycle changes. **SharePoint as the canonical SOP store** lacks the same validated quality-document workflows in organizations where Vault is the record system.

4. DISCUSSION POINTS

- API quotas and large binary renditions require paging, resumable downloads, and backoff; a full rescan should not be the normal recovery path.
- Draft, obsolete, and superseded SOPs must not silently enter current-answer retrieval; retain them for audit with lifecycle filters.
- Never flatten ACLs during export. The retrieval layer must enforce document authorization before text reaches an LLM.

TrackWise — QMS records

Adds point-in-time site risk from the quality system while preserving record history.

1. ROLE

Brings deviations, observations, corrective and preventive actions (CAPAs), investigations, and effectiveness checks into the facility-risk context. It supplies the operational evidence that turns a general obligation into a site-specific inspection concern.

2. TECHNOLOGY RATIONALE

The primitive is incremental extraction from a quality-management transactional system with slowly changing records and status histories. Use source modification timestamps plus a monotonically tracked watermark, but periodically reconcile by business key because timestamps can be corrected. CDC-like semantics are desirable; delivery to the lake remains at-least-once, so record versions and idempotent upserts are required.

3. WHY THIS OVER ALTERNATIVES

Streaming every database log change offers low latency but is often unavailable for SaaS deployments and overstates the need—hourly freshness is usually adequate. **Nightly full exports** are easier but scale poorly and make deletion or correction semantics ambiguous. **Using only public 483 data** misses the company-specific leading indicators that make the prediction useful.

4. DISCUSSION POINTS

- CAPA text can contain sensitive quality, employee, or patient information; tokenize or redact before external model calls.
- Status changes are not append-only. Preserve history so training features can be reconstructed point-in-time without leakage.
- A senior interviewer may challenge whether open CAPA count is a causal signal; present it as a ranking feature with temporal validation, not a compliance conclusion.

SharePoint — LMS and files

Uses delta-aware enterprise content access while retaining versions and source permissions.

1. ROLE

Captures less-structured enterprise content such as local procedures, training exports, inspection playbooks, and supporting files. It extends coverage where content has not yet migrated into validated systems.

2. TECHNOLOGY RATIONALE

The primitive is graph/API-based hierarchical content access with delta tokens, file versioning, and principal-based permissions. Delta queries minimize rescans, while drive-item IDs and ETags support idempotency. The connector is eventually consistent with tenant indexing and must copy source ACLs into the retrieval metadata rather than trusting SharePoint search at query time.

3. WHY THIS OVER ALTERNATIVES

Mounting SharePoint as a network drive hides version and permission metadata and is brittle at scale. **Manual uploads** cannot meet freshness or lineage requirements. **Microsoft Graph search alone** is unsuitable as the RAG backend because ranking, retention, and availability are controlled outside this system.

4. DISCUSSION POINTS

- Delta tokens expire and permission inheritance can change without document content changing; schedule ACL reconciliation.
- File formats, encrypted files, and links to external tenants need quarantine paths rather than silent drops.
- The trap is treating all SharePoint content as authoritative. Carry source-system, owner, lifecycle, and confidence fields into ranking.

Amazon EventBridge Scheduler

Serverless, independently managed, at-least-once scheduling without scheduler-host operations.

1. ROLE

Triggers each source connector on its own cadence and launches periodic reconciliation, reprocessing, and precompute jobs. Schedules are configuration, not hard-coded timers inside workers.

2. TECHNOLOGY RATIONALE

The primitive is managed time-based invocation with retry policy, flexible time windows, and decoupled targets. It is serverless, horizontally managed, and appropriate for hundreds of low-frequency schedules; delivery is at-least-once, so every target must be idempotent. It removes the need to keep a scheduler host alive and provides explicit operational ownership per source.

3. WHY THIS OVER ALTERNATIVES

Cron on EC2 introduces a single host, patching, failover, and clock-drift burden. **Apache Airflow / MWAA** is justified for complex dependency DAGs but expensive and operationally heavy for independent polling triggers. **Step Functions wait states** can schedule one workflow, but are awkward for a fleet of independently configured recurring jobs.

4. DISCUSSION POINTS

- Schedules may fire late or more than once; use a run key such as source plus scheduled window and make connector writes conditional.
- Avoid one global cadence: high-churn enforcement feeds and quarterly guidance archives have different freshness and cost profiles.
- Watch per-region quotas and retry storms after an outage; jitter and per-source concurrency caps prevent synchronized bursts.

AWS Lambda — source connectors

Elastic stateless compute for short I/O-bound connectors, with concurrency used as a safety valve.

1. ROLE

Executes short-lived fetch, validation, normalization, and manifest-writing logic for each external source. A connector emits raw bytes and metadata, not interpreted regulatory conclusions.

2. TECHNOLOGY RATIONALE

The primitive is event-driven stateless compute with rapid horizontal scaling, per-invocation isolation, and pay-per-use economics. It fits I/O-bound polling jobs that finish within 15 minutes and need modest memory. Reserved concurrency bounds traffic to fragile sources; retry behavior integrates with SQS and EventBridge, while small deployment packages keep cold starts manageable.

3. WHY THIS OVER ALTERNATIVES

AWS Fargate handles long downloads and custom runtimes better but incurs task startup latency and more orchestration for tiny polls. **EC2 workers** offer total control but waste capacity between runs and add patching. **AWS Glue jobs** are optimized for distributed data transformation, not lightweight HTTP connectors.

4. DISCUSSION POINTS

- The 15-minute execution limit, ephemeral disk, response-size limits, and NAT data-processing charges are real constraints; route oversize work to Fargate.
- Source throttling must be enforced with reserved concurrency and token buckets; Lambda's scale is a liability against rate-limited regulators.
- Separate retrieval from parsing so a bad parser can be replayed against S3 without refetching the source.

Amazon SQS with dead-letter queue

Durable at-least-once work buffering that absorbs bursts and makes poison jobs recoverable.

1. ROLE

Buffers discovered document work between connectors and downstream parsing/diff stages. The dead-letter queue (DLQ) isolates messages that repeatedly fail without blocking healthy work.

2. TECHNOLOGY RATIONALE

The primitive is a durable pull queue with at-least-once delivery, visibility timeouts, long polling, and backpressure. Standard queues maximize throughput; consumers must deduplicate by document-version key. A DLQ plus redrive policy makes poison messages inspectable and replayable, while queue depth provides a direct lag signal.

3. WHY THIS OVER ALTERNATIVES

Amazon Kinesis Data Streams is better when ordered replayable event logs and many independent consumers are required, but partition-key management is unnecessary for document jobs. **Amazon MSK/Kafka** provides rich retention and stream semantics at far greater operational cost. **Direct Lambda invocation** has no durable buffer or backpressure boundary.

4. DISCUSSION POINTS

- Set visibility timeout above worst-case processing plus heartbeat extension; otherwise one long PDF produces duplicate work.
- A DLQ is not a garbage bin: alarm on age and count, store failure reason, and define a tested redrive procedure.
- FIFO queues offer ordered, deduplicated delivery but lower throughput and a five-minute deduplication window; use them only if ordering is a real invariant.

Amazon S3 raw lake with Object Lock

Immutable, versioned, replayable evidence with eleven-nines durability and lifecycle economics.

1. ROLE

Stores the exact source bytes, response metadata, normalized text, and version manifests before interpretation. It is the replay source and defensible audit record for what the system saw at a given time.

2. TECHNOLOGY RATIONALE

The primitive is massively durable object storage with read-after-write consistency, versioning, lifecycle tiers, checksum validation, and Write Once Read Many (WORM) retention through Object Lock. Object keys partition by source/date/document/version; content hashes make writes idempotent. Storage scales independently of compute and supports reprocessing years later.

3. WHY THIS OVER ALTERNATIVES

EFS provides POSIX semantics but costs more at archive scale and does not naturally provide immutable object versions. **DynamoDB blobs** have item-size limits and poor economics for PDFs. A **conventional database** is useful for manifests, not immutable multi-megabyte source payloads.

4. DISCUSSION POINTS

- Object Lock requires versioning and careful governance-mode versus compliance-mode selection; retention mistakes can become legally undeletable and costly.
- Lifecycle transitions save money but increase restore latency; keep recent material in Standard or Intelligent-Tiering and archive older raw bytes deliberately.
- Encryption, legal hold, cross-region replication, and deletion policy must align with records management—not just engineering preference.

AWS Step Functions — diff pipeline

Durable, inspectable orchestration for multi-step, retryable, human-reviewable change processing.

1. ROLE

Orchestrates normalization, version lookup, textual alignment, semantic review, classification, persistence, and alert routing with explicit branches and compensation paths.

2. TECHNOLOGY RATIONALE

The primitive is durable workflow state with per-step retries, timeouts, parallel maps, human-wait callbacks, and execution history. Standard Workflows provide exactly-once workflow execution semantics for up to one year, while activities may still cause external side effects more than once and must be idempotent. The visual state machine makes high-consequence decision paths auditable.

3. WHY THIS OVER ALTERNATIVES

Apache Airflow / MWAA is stronger for scheduled batch DAGs and backfills but slower and heavier for event-triggered per-document workflows. **Lambda choreography through events** is cheaper at first yet hides state and error ownership. **A monolithic Fargate worker** makes partial retries and human review awkward.

4. DISCUSSION POINTS

- State-transition pricing can dominate when a Map state expands thousands of tiny chunks; batch work or use Express selectively.
- Execution history contains inputs and outputs unless constrained; pass S3 pointers, not sensitive document bodies.
- Exactly-once workflow execution does not mean exactly-once downstream writes—use conditional updates and idempotency keys.

OpenAI materiality judge

Schema-constrained semantic judgment for ambiguous diffs, with recall-first human escalation.

1. ROLE

Classifies aligned old/new passages as editorial or material and emits a constrained explanation, change type, affected topic, and confidence. It handles semantic changes that line-level diffs cannot reliably interpret.

2. TECHNOLOGY RATIONALE

The primitive is schema-constrained large-language-model reasoning over a bounded evidence window. The call is asynchronous from ingestion, protected by strict timeouts, versioned prompts, deterministic settings where possible, and a fallback queue. It optimizes recall: uncertain or high-impact results route to humans rather than being dropped. Model output is advisory until grounded against the diff spans.

3. WHY THIS OVER ALTERNATIVES

Rule-based keyword detection is transparent but misses negation, scope, and meaning-preserving rewrites. **A fine-tuned BERT classifier** can deliver lower latency and cost once enough labels exist, but initially lacks broad semantic coverage and explanations. **Embedding-distance thresholds** detect change magnitude, not regulatory materiality.

4. DISCUSSION POINTS

- External API latency, quota, regional routing, zero-data-retention terms, and data classification must be explicit architectural assumptions.
- Prompt injection inside source documents is data, not instruction; isolate quoted text, use strict schemas, and never grant the model tools.
- Measure false negatives by expert adjudication. Confidence is not calibrated probability; thresholding needs a held-out labeled set.

Amazon DynamoDB — change registry

Low-latency idempotency and version-state registry with conditional writes and no fleet to manage.

1. ROLE

Indexes each canonical document version, hashes, processing status, impact score, applicable entities, and pointers to S3 evidence. It supplies low-latency idempotency checks and workflow state lookups.

2. TECHNOLOGY RATIONALE

The primitive is a serverless key-value/document database with single-digit-millisecond access, conditional writes, transactions for bounded items, streams, and on-demand capacity. A document ID partition key plus version sort key supports history; conditional expressions prevent duplicate version registration. Reads are eventually consistent by default and can be strongly consistent where the workflow requires read-after-write truth.

3. WHY THIS OVER ALTERNATIVES

Amazon Aurora PostgreSQL is better for ad hoc joins and complex relational integrity, but needs connection/capacity management and is unnecessary for key-centric workflow state. **S3 manifests alone** are durable but slow for conditional state transitions and secondary lookups. **OpenSearch** is a search projection, not a durable source of workflow truth.

4. DISCUSSION POINTS

- Poor partition keys can create hot keys when many chunks update one document; aggregate status separately or shard child items.
- Global secondary indexes are eventually consistent and add write cost; design only the access patterns actually needed.
- The 400 KB item limit means storing pointers and summaries, never full text or large model responses.

Amazon SNS — impact alerts

At-least-once filtered fan-out after durable commit, with SQS used for dependable machine consumers.

1. ROLE

Fans material-change notifications to site/topic subscriptions, operational queues, email channels, and downstream recompute triggers after the change registry commits.

2. TECHNOLOGY RATIONALE

The primitive is managed publish/subscribe fan-out with filter policies and at-least-once delivery. One published event can reach SQS queues, Lambda, HTTP endpoints, or human notification paths; durable machine processing should generally subscribe through SQS. Publishing only after durable state commit keeps alert consumers able to dereference a complete record.

3. WHY THIS OVER ALTERNATIVES

Amazon EventBridge offers richer content routing, schema discovery, and cross-account buses but has higher per-event cost and is unnecessary for simple topic fan-out. **Direct email from Lambda** couples workflow completion to a fragile channel and cannot feed machine consumers. **Kafka** is excessive for low-volume alert distribution.

4. DISCUSSION POINTS

- Email/SMS endpoints can be delayed or duplicated; alerts are notifications, not the system of record.
- Filter-policy drift can silently miss a site; test subscriptions against the facility/topic taxonomy and alarm on undelivered events.
- Avoid alert fatigue with severity thresholds, digesting, and ownership. “Sent” is not “acknowledged”; critical alerts need escalation state elsewhere.

Amazon Textract — OCR and layout

Managed layout-aware OCR that retains coordinates and confidence for traceable citations.

1. ROLE

Converts scanned PDFs and forms into text, reading order, tables, key-value pairs, page coordinates, and confidence values. It preserves document structure needed for section-aware chunks and defensible citations.

2. TECHNOLOGY RATIONALE

The primitive is managed asynchronous optical character recognition (OCR) and document-layout analysis. Async jobs handle multi-page documents and publish completion events; outputs retain bounding boxes so a citation can point back to the rendered page. The service scales without GPU fleet management, and confidence scores create a review threshold instead of treating recognition as certain.

3. WHY THIS OVER ALTERNATIVES

Tesseract on Fargate is cheaper at sustained scale and fully controllable, but requires image preprocessing, language packs, model tuning, and capacity operations; its layout/table quality is weaker out of the box. **Amazon Rekognition** detects text in images but is not designed for multipage document structure. **LLM vision parsing** is flexible but costlier and less deterministic for bulk OCR.

4. DISCUSSION POINTS

- Native digital PDFs should use text extraction first; OCR only when needed to avoid cost and recognition errors.
- Scans with stamps, rotated pages, handwriting, merged cells, and tiny footnotes need confidence-based human review.
- Textract output order is not guaranteed to match semantic reading order; rebuild sections from geometry and heading signals.

AWS Fargate — section-aware chunking

Container flexibility and bounded concurrency for variable, long-running document parsing jobs.

1. ROLE

Runs parsers that reconstruct headings, lists, tables, references, and semantic sections, then emits stable chunks with token counts and source coordinates.

2. TECHNOLOGY RATIONALE

The primitive is serverless container compute: arbitrary libraries, CPU/memory sizing, ephemeral storage, and jobs longer than Lambda's 15-minute cap without managing EC2 hosts. Chunking is CPU- and memory-variable across PDFs, so SQS-driven Fargate tasks allow bounded concurrency and independent scaling. Container digests make parser behavior reproducible across reprocessing runs.

3. WHY THIS OVER ALTERNATIVES

AWS Lambda is attractive for small files but constrained by duration, package size, ephemeral storage, and burst behavior. **AWS Glue** is optimized for distributed tabular ETL, not layout-heavy per-document parsing. **Amazon EKS** provides sophisticated scheduling but adds cluster operations without a demonstrated need.

4. DISCUSSION POINTS

- Task startup latency is acceptable for asynchronous preparation but not interactive queries; batch several tiny documents if launch cost dominates.
- Stable chunk IDs should derive from document version plus normalized section path and span, so reindexing is idempotent.
- Naive fixed-token chunking breaks obligations and tables. Discuss semantic boundaries, overlap, parent-child retrieval, and token-budget trade-offs.

OpenAI structured obligation extraction

Converts complex normative language into typed, span-grounded obligations before graph loading.

1. ROLE

Extracts normative statements into typed records: subject, modality, action, object, conditions, exceptions, dates, and exact source span. The result feeds the graph, templates, and applicability reasoning.

2. TECHNOLOGY RATIONALE

The primitive is schema-constrained generative information extraction over a small, source-grounded section. Structured outputs reduce parser ambiguity; span-verification rejects any field that cannot be traced to source text. Calls are batchable, independently retryable, prompt/model-versioned, and routed to human review for high-impact or low-confidence cases.

3. WHY THIS OVER ALTERNATIVES

Regex and dependency parsing are cheap and explainable but fail on cross-sentence scope, exceptions, and nested conditions. **Fine-tuned token classification** can become the production choice with a large labeled corpus, yet requires ontology-stable annotations and still struggles to normalize relationships. **Manual-only extraction** is high quality but too slow for broad coverage and change latency.

4. DISCUSSION POINTS

- Distinguish “shall,” “must,” “should,” and descriptive text; modality errors materially change risk.
- Require exact source spans and validate IDs. A plausible obligation without evidence is a pipeline defect, not a low-confidence record.
- Version the extraction schema and ontology; changes require backfill, dual-read compatibility, and regression evaluation.

Amazon Neptune — knowledge graph

Provides bounded multi-hop applicability and provenance that vector similarity cannot guarantee.

1. ROLE

Represents typed relationships among regulations, sections, obligations, topics, processes, products, facilities, inspections, observations, CAPAs, and questions. It answers multi-hop applicability and provenance queries used as hard ranking features.

2. TECHNOLOGY RATIONALE

The primitive is a managed graph database supporting property-graph traversal (Gremlin/openCypher) and RDF/SPARQL models. The workload is relation-heavy: “facility manufactures product through process governed by obligation amended by change” is a bounded traversal, not a text-similarity problem. Neptune provides replicas, backups, and transaction semantics without operating a graph cluster directly.

3. WHY THIS OVER ALTERNATIVES

Aurora PostgreSQL can model edges and recursive CTEs, but deep or evolving traversals become join-heavy and harder to maintain. **OpenSearch vectors** retrieve semantically similar text but cannot guarantee typed applicability paths. **Neo4j self-managed** has a strong ecosystem yet adds cluster, patching, and recovery responsibility on AWS.

4. DISCUSSION POINTS

- Supernodes such as broad regulations can produce expensive fan-out; constrain traversal depth, label filters, and edge direction.
- Entity resolution is the real difficulty: aliases, superseded sections, and process taxonomy need deterministic IDs and steward review.
- Graph truth and search projections can diverge; use versioned events, reconciliation jobs, and explicit effective-time edges.

OpenAI embeddings — text-embedding-3-large

High-recall semantic representation, versioned as carefully as any learned model.

1. ROLE

Encodes chunks, obligations, questions, and serialized contexts into vectors used for semantic retrieval, deduplication, clustering, and similarity features.

2. TECHNOLOGY RATIONALE

The primitive is a dense semantic representation with cross-domain language coverage. Large embeddings support high recall on technical regulatory phrasing; dimensionality may be shortened only after retrieval-quality evaluation. Offline batch calls amortize cost, while every vector carries model ID, dimensions, normalization policy, source version, and content hash for reproducibility.

3. WHY THIS OVER ALTERNATIVES

Amazon Titan Text Embeddings keeps inference within AWS and may simplify governance, but the deciding factor must be measured domain recall, not vendor preference. **Sentence Transformers hosted on SageMaker** offer control and lower marginal cost at scale, but require model serving, upgrades, and domain tuning. **BM25 alone** is excellent for citations and acronyms but misses paraphrases and concept-level similarity.

4. DISCUSSION POINTS

- An embedding upgrade is a schema migration: dual-index, backfill, evaluate, and switch atomically; never mix incompatible vector spaces.
- Vector similarity is not authority or applicability. Metadata filters and graph checks remain mandatory.
- Track token cost, batching limits, rate limits, and cross-region data movement; cache by normalized content hash.

OpenSearch Serverless — hybrid BM25 and vector index

Managed hybrid lexical-plus-vector search, with ACL prefilters and fusion for robust recall.

1. ROLE

Serves ACL-filtered retrieval across text chunks using lexical BM25 and approximate nearest-neighbor vector search, then fuses results for candidate and evidence retrieval.

2. TECHNOLOGY RATIONALE

The primitive is managed distributed search with inverted indexes, HNSW-style k-nearest-neighbor search, metadata filters, and independently scaled indexing/query capacity. Lexical retrieval protects exact regulation numbers, product codes, and acronyms; vectors recover paraphrases. Reciprocal-rank fusion avoids comparing incomparable raw score scales and gives robust hybrid recall.

3. WHY THIS OVER ALTERNATIVES

Aurora PostgreSQL with pgvector is simpler for modest corpora and transactional joins, but distributed full-text plus ANN scaling and filter performance become harder at tens of millions of chunks. **Pinecone** is operationally simple for vectors but adds another data plane and still needs lexical search. **Self-managed Elasticsearch** exposes tuning control at a much higher operations burden.

4. DISCUSSION POINTS

- Serverless has minimum capacity and burst/cold behavior; benchmark actual filter selectivity and concurrency before assuming cheaper economics.
- HNSW memory grows with vectors and graph parameters; dimensionality, ef_search, and recall/latency require empirical tuning.
- ACL filtering must occur during retrieval, not after top-k, or unauthorized documents affect rankings and can leak through answers.

SageMaker Feature Store — point-in-time features

Unifies low-latency serving with leakage-safe historical joins to reduce training-serving skew.

1. ROLE

Serves facility, process, risk, inspection, and recent-change features for online ranking while preserving historical values for leakage-free training joins.

2. TECHNOLOGY RATIONALE

The primitive is a dual online/offline feature repository with consistent feature definitions, record timestamps, and low-latency key lookup. The online store supports millisecond inference reads; the S3-backed offline store supports point-in-time joins so a 2024 training example cannot see a CAPA opened in 2025. Feature ownership and lineage reduce training-serving skew.

3. WHY THIS OVER ALTERNATIVES

DynamoDB plus custom ETL can serve online values cheaply but requires hand-built historical snapshots, joins, registry, and lineage. **Redis alone** is fast but unsuitable as a durable offline feature history. **Feast** is portable and capable, but adds control-plane operations unless multi-cloud portability is a hard requirement.

4. DISCUSSION POINTS

- Event time, ingestion time, and observation time differ; point-in-time correctness depends on using the right timestamp.
- Online values can become stale even when the store is healthy; publish freshness per feature family and define defaults explicitly.
- Do not place high-cardinality text or document payloads here. Feature Store holds stable numeric/categorical features and references.

Candidate generation — ANN, LLM, templates

Combines proven history, controlled novelty, and deterministic coverage to maximize candidate recall.

1. ROLE

Builds a broad, deduplicated candidate set from three complementary paths: retrieval of proven questions, grounded generation of novel questions, and deterministic coverage of each obligation.

2. TECHNOLOGY RATIONALE

The primitive is multi-retriever recall maximization before precision ranking. ANN search returns historically relevant questions in tens of milliseconds; the LLM expands beyond history under an obligation-ID schema; templates guarantee coverage and deterministic fallback. Results carry provenance and generator type, then cluster/deduplicate before the ranker.

3. WHY THIS OVER ALTERNATIVES

LLM-only generation offers novelty but is slower, costlier, less repeatable, and vulnerable to unsupported questions. **Retrieval-only** is fast and grounded but cannot cover new regulations or cold-start contexts. **One giant classifier over all questions** scales poorly as the label set changes and cannot create new questions without retraining.

4. DISCUSSION POINTS

- Set per-path quotas so popular retrieval candidates do not crowd out new obligations; measure recall contribution and marginal gain by source.
- Generation is asynchronous and optional for serving: cached retrieval/template candidates keep the system available during model outages.
- Deduplication thresholds can collapse meaningfully different questions; cluster within topic and preserve source obligations.

LambdaMART ranker with MMR diversification

Fast interpretable learning-to-rank for heterogeneous signals, with MMR protecting list-level coverage.

1. ROLE

Scores each context-question pair with learned nonlinear feature interactions, then applies maximal marginal relevance (MMR) so the top list covers distinct risks rather than repeating one topic.

2. TECHNOLOGY RATIONALE

The primitive is gradient-boosted learning-to-rank optimized for ordered lists through pairwise LambdaRank gradients. LambdaMART handles mixed numeric/categorical signals, missing values, nonlinear thresholds, and feature attribution with low inference latency. MMR is a deterministic post-ranker balancing relevance against semantic redundancy; calibrated scores support stable thresholds, not causal probabilities.

3. WHY THIS OVER ALTERNATIVES

Weighted heuristics are transparent and useful as a baseline but become brittle as signals interact. **A neural cross-encoder ranker** may improve semantic precision but raises latency, cost, and explainability burden for hundreds of candidates. **Pure cosine similarity** ignores facility risk, recency, inspector affinity, and obligation criticality.

4. DISCUSSION POINTS

- Training labels are exposure-biased: shown questions are more likely to be logged. Use randomized exploration or inverse-propensity weighting and report assumptions.
- Temporal splits and site-held-out tests are mandatory; random splits leak recurring question clusters.
- MMR lambda is a product trade-off. Too much diversity suppresses genuinely concentrated risk; tune against hit-rate and coverage together.

Amazon ElastiCache — context-hash cache

Permission-aware low-latency reuse of expensive inference artifacts, with TTL as a correctness bound.

1. ROLE

Caches precomputed candidate lists, ranked results, retrieval outputs, and answer status under a deterministic hash of context, permissions, and model versions.

2. TECHNOLOGY RATIONALE

The primitive is managed in-memory key-value access with sub-millisecond-to-low-millisecond reads, expiration, clustering, and optional replication. It removes repeated ANN and ranking work from interactive requests. Keys include feature snapshot, regulation version, ACL scope, prompt/model version, and already-asked set; event-driven invalidation supplements a bounded TTL.

3. WHY THIS OVER ALTERNATIVES

DynamoDB DAX accelerates DynamoDB reads but does not cache arbitrary multi-stage inference artifacts cleanly. **Application-local caches** are fastest but fragment across tasks and disappear on deployment. **CloudFront** is useful for public HTTP objects, not permission-sensitive dynamic ranking state.

4. DISCUSSION POINTS

- Cache keys that omit tenant, role, or ACL version can create a security incident. Prefer false misses over cross-user hits.
- Invalidation events can be lost or delayed; TTL is the correctness backstop, not just cost control.
- Redis cluster resharding, hot keys, memory eviction, and replica failover can spike latency; cache misses must degrade gracefully.

OpenAI grounded RAG — citations and NLI gate

Citation-constrained synthesis with independent entailment checks and an explicit abstention path.

1. ROLE

Generates a concise draft answer and evidence pack for each high-ranked question from ACL-filtered retrieved passages, then verifies sentence-level support before showing it.

2. TECHNOLOGY RATIONALE

The primitive is retrieval-augmented generation (RAG) with constrained citation output followed by a natural-language-inference (NLI) entailment gate. Retrieval and authorization happen before model input; every claim points to document ID, section, page, and span. Failed entailment triggers one repair pass, then sentence removal or abstention. Answers stream after questions so generation latency never blocks the ranked list.

3. WHY THIS OVER ALTERNATIVES

Fine-tuning the LLM on SOPs does not provide current facts, deletions, ACL enforcement, or citations. **Direct long-context prompting** is simple but expensive and dilutes relevant evidence as corpora grow. **Extractive-only answers** maximize faithfulness but are less useful when evidence must be synthesized across several documents.

4. DISCUSSION POINTS

- NLI models can approve lexical overlap without true support and reject valid domain inference; calibrate on expert-labeled regulatory claims.
- Citation correctness and answer correctness are separate metrics. Verify that the cited span exists, is accessible, current, and entails the sentence.
- Define abstention as a successful outcome. The interview trap is optimizing answer rate instead of harmful unsupported-answer rate.

Amazon API Gateway with AWS WAF

Managed API control plane that protects and meters scarce downstream inference capacity.

1. ROLE

Exposes authenticated prediction, feedback, and evidence endpoints to the Copilot UI while enforcing request validation, throttling, routing, and perimeter protections.

2. TECHNOLOGY RATIONALE

The primitive is a managed API front door with TLS termination, authorization integration, quotas, request/response controls, observability, and WebSocket support. WAF adds managed and custom rules for injection patterns, IP reputation, and rate abuse. The gateway separates public API lifecycle from Fargate/Lambda implementation and protects scarce downstream model capacity.

3. WHY THIS OVER ALTERNATIVES

Application Load Balancer is efficient for high sustained HTTP traffic and long-lived services, but lacks the same API-key, usage-plan, request-model, and WebSocket management features. **CloudFront plus Lambda URLs** can be cheaper but requires assembling more auth and throttling controls. **Direct service exposure** unnecessarily expands the attack surface.

4. DISCUSSION POINTS

- API Gateway has payload, timeout, and WebSocket semantics that make long model calls unsuitable; return job IDs or stream through dedicated channels.
- WAF cannot authorize documents or stop prompt injection in retrieved content; it is network/request protection, not application trust.
- Protect downstream cost with per-tenant quotas, maximum context size, and idempotency keys—not only generic rate limits.

Precompute worker — nightly and event-driven

Materializes expensive predictions ahead of need, with events for freshness and nightly repair for completeness.

1. ROLE

Builds each facility's top candidate/ranked question set before an inspection and refreshes affected sites when regulations, CAPAs, permissions, or models change.

2. TECHNOLOGY RATIONALE

The primitive is asynchronous materialization of expensive features and ranked views. Event-driven jobs minimize staleness after important changes; nightly reconciliation repairs missed events and model-wide updates. Work is partitioned by facility/context, checkpointed, idempotent, and written behind a versioned pointer so readers never observe a partial result.

3. WHY THIS OVER ALTERNATIVES

Fully on-demand computation simplifies freshness semantics but exposes users to LLM and retrieval tail latency during a surprise inspection. **Nightly-only batch** is cheap but can miss urgent same-day changes. **Always-running streaming computation** offers lower latency than the use case requires and increases stateful operations.

4. DISCUSSION POINTS

- A new model or embedding version can trigger a thundering-herd backfill; prioritize high-risk sites and use bounded queues.
- Precompute must include an “as of” timestamp and context fingerprint so the UI can disclose staleness.
- If permissions change, invalidate affected results immediately; nightly recovery is not adequate for access-control correctness.

Analyst Copilot UI

Task-oriented inspection workflow that exposes rationale and captures position-aware feedback.

1. ROLE

Presents the top predicted questions, rationale, evidence readiness, cited draft answers, and an inspection-mode workflow for logging what was actually asked.

2. TECHNOLOGY RATIONALE

The primitive is a task-oriented web client over versioned APIs, not a chat box. It separates ranked questions from asynchronously arriving answers, preserves source/citation affordances, and emits explicit interaction events with prediction IDs and exposure positions. Local resilience may retain the current inspection list briefly, while sensitive evidence remains server-authorized.

3. WHY THIS OVER ALTERNATIVES

A generic chat interface hides coverage, rank order, evidence gaps, and negative feedback. **A static dashboard** cannot adapt after each asked question or capture useful labels. **A mobile-native app** can improve offline behavior but raises distribution and device-management cost before the interaction model is proven.

4. DISCUSSION POINTS

- Show “why predicted,” “why now,” source age, and evidence gaps; confidence without rationale encourages automation bias.
- Logging must distinguish displayed, opened, copied, marked asked, dismissed, and corrected—these are not equivalent labels.
- Design for degraded mode: cached questions may remain visible during outages, but stale evidence and new answers must be clearly disabled.

Amazon Kinesis Data Firehose

Managed, buffered, at-least-once delivery of interaction telemetry into durable analytics storage.

1. ROLE

Delivers prediction exposures, clicks, asked-question events, corrections, latency, and model metadata into the analytics lake with buffering and managed retry.

2. TECHNOLOGY RATIONALE

The primitive is managed delivery streaming: producers emit records, Firehose batches by size/time, optionally transforms/compresses them, and writes partitioned objects to S3. Delivery is at-least-once, so events carry immutable event IDs and downstream jobs deduplicate. It removes shard management where the main requirement is reliable landing, not subsecond multi-consumer processing.

3. WHY THIS OVER ALTERNATIVES

Kinesis Data Streams is preferable for many independent real-time consumers and ordered replay, but requires shard/capacity management that this landing path does not need. **Amazon MSK** gives long retention and rich stream processing at much higher operational cost. **Direct UI writes to S3** are insecure and create tiny-object and retry problems.

4. DISCUSSION POINTS

- Buffering means feedback is not immediately visible; define the acceptable freshness for dashboards and retraining separately.
- Malformed records should land in an error prefix with schema/version and reason; silently dropping them biases training.
- Small-file economics, transformation Lambda failures, and dynamic partition limits need load testing.

S3 Parquet event lake with AWS Glue catalog

Open, durable, columnar event history for reproducible analytics and model training.

1. ROLE

Stores immutable interaction, exposure, label, and model-lineage events in columnar partitions and exposes governed schemas to Athena, training jobs, and dashboards.

2. TECHNOLOGY RATIONALE

The primitive is an open-format analytical lake: S3 provides durable low-cost storage; Parquet provides column pruning, compression, and predicate pushdown; Glue stores table/partition metadata. Partition by event date and coarse tenant/site boundaries—not high-cardinality IDs—and compact Firehose output to avoid tiny files. Append-only events support reproducible label rebuilding.

3. WHY THIS OVER ALTERNATIVES

Amazon Redshift excels at governed interactive warehouses but costs more for long-retention raw events and couples data to warehouse loading. **DynamoDB** is poor for scans and analytical joins. **JSON in S3** is easy to land but expensive to scan and weakly typed for repeated training jobs.

4. DISCUSSION POINTS

- Schema evolution needs compatibility rules; required-field changes should create a new version rather than corrupt old readers.
- Late and duplicate events require watermarking and event-ID deduplication during silver/gold table construction.
- Avoid partitioning by user or prediction ID. Millions of tiny prefixes and files destroy query and training throughput.

SageMaker Pipelines — retraining

Reproducible ML-native workflow with lineage and conditional quality gates before registration.

1. ROLE

Orchestrates label construction, point-in-time feature joins, training, calibration, offline evaluation, bias/slice analysis, and conditional registration of a new ranker.

2. TECHNOLOGY RATIONALE

The primitive is a versioned machine-learning workflow with cached steps, managed processing/training jobs, lineage, parameterization, and quality gates. Each run binds dataset snapshot, feature definitions, code image, hyperparameters, and evaluation report. Conditional transitions stop a model that improves aggregate nDCG while harming critical slices.

3. WHY THIS OVER ALTERNATIVES

Step Functions can orchestrate ML jobs but needs custom lineage, experiment, and model-registration plumbing. **Airflow / MWAA** is flexible across ecosystems but requires more operator work for SageMaker-native metadata and artifacts. **Notebook-driven retraining** is irreproducible and cannot be trusted for promotion.

4. DISCUSSION POINTS

- Retraining cadence should follow label volume and drift, not a ritual weekly schedule; low-volume sites may need pooled models.
- Pipeline success only means the jobs ran. Promotion requires temporal metrics, site/trigger slices, calibration, and regression thresholds.
- Pin container digests, dependency locks, random seeds, feature versions, and input manifests; reproducibility is an operational feature.

SageMaker Model Registry — shadow, canary, production

Governed, lineage-rich promotion separates model creation from safe operational release.

1. ROLE

Versions model artifacts and approval state, then controls promotion through shadow evaluation, limited canary traffic, and full production rollout with rollback.

2. TECHNOLOGY RATIONALE

The primitive is a governed model package catalog containing artifact URI, container, inference specification, metrics, lineage, and approval status. Deployment stages are externalized from training: shadow observes without user impact, canary receives a bounded fraction, and production is an explicit approval. Model and feature/schema compatibility are promoted together.

3. WHY THIS OVER ALTERNATIVES

Storing model binaries directly in S3 preserves bytes but not approval workflow, lineage, compatibility, or deployment state. **ECR tags alone** are mutable pointers and describe containers, not model evaluation. **MLflow** is a strong portable registry but introduces another service unless portability or existing adoption outweighs native integration.

4. DISCUSSION POINTS

- Shadow traffic cannot measure behavioral lift if users never see results; combine offline replay with online canary evidence.
- Canary routing must be sticky by inspection/session to avoid lists changing mid-event.
- Rollback must include model, feature contract, cache namespace, and calibration—not merely the model file.

Amazon CloudWatch — hit-rate@10 and operations

Unifies operational telemetry with cohort-aware product quality metrics and alarms.

1. ROLE

Collects service metrics, structured logs, traces, and business-quality signals such as hit-rate@10, evidence latency, abstention rate, and per-stage failures.

2. TECHNOLOGY RATIONALE

The primitive is managed telemetry aggregation with dimensions, dashboards, metric math, logs, traces, and alarms. Technical metrics arrive near-real-time; quality metrics such as hit-rate@10 are delayed until actual questions are logged and should carry model/context cohort dimensions. Embedded metric format avoids a synchronous PutMetric call per event.

3. WHY THIS OVER ALTERNATIVES

Prometheus/Grafana provides rich portable metrics but adds scraping/storage operations and is less natural for asynchronous AWS jobs. **Datadog** offers excellent cross-stack UX at added vendor and ingestion cost. **QuickSight alone** is for analysis, not low-latency operational alerting.

4. DISCUSSION POINTS

- High-cardinality dimensions such as user, document, or prediction ID create cost and unusable metric cardinality; keep those in logs/lake.
- Hit-rate@10 is exposure- and logging-dependent; pair it with coverage, calibration, abstention, and evidence quality.
- Set service-level objectives per stage. A green API latency chart can hide a failed ingestion feed for days.

AWS Key Management Service (KMS)

Centralized envelope encryption and auditable key policy without customer-operated HSMs.

1. ROLE

Provides envelope-encryption keys and policy controls for S3, DynamoDB, OpenSearch, logs, queues, snapshots, and secrets across the system.

2. TECHNOLOGY RATIONALE

The primitive is a regional hardware-security-module-backed key service. AWS services generate data keys and encrypt data locally; KMS protects the small data keys, enabling centralized policy, rotation, grants, and audit without routing every byte through KMS. Separate customer-managed keys by environment and major data classification boundary to limit blast radius.

3. WHY THIS OVER ALTERNATIVES

AWS-owned keys are simplest but offer less policy control and account-level separation. **CloudHSM** gives exclusive HSM control but shifts cluster availability and key operations to the customer.

Application-managed encryption keys invite rotation, storage, and audit failures.

4. DISCUSSION POINTS

- KMS request quotas and per-request charges can appear in high-volume object or log workflows; envelope encryption and bucket keys reduce calls.
- Key policy plus IAM policy is a common lockout trap; test break-glass recovery and cross-account grants.
- Deletion is destructive after the waiting period. Prefer disable-first procedures with dependency inventory.

AWS Identity and Access Management (IAM)

Short-lived workload identities and policy-enforced least privilege at every AWS boundary.

1. ROLE

Defines workload identities, human roles, and least-privilege authorization across every service call and data plane boundary.

2. TECHNOLOGY RATIONALE

The primitive is signed, short-lived, policy-evaluated authorization using roles and AWS Security Token Service (STS), with explicit denies, conditions, resource policies, and permission boundaries. Each connector, processor, trainer, and serving task gets a separate role; session tags can propagate tenant or environment context. No long-lived static AWS keys are required.

3. WHY THIS OVER ALTERNATIVES

Shared IAM users/access keys are easy initially but hard to rotate, attribute, and contain. **Application-only RBAC** cannot prevent a compromised service from calling AWS resources directly.

Kubernetes service accounts can federate to IAM on EKS, but EKS is not otherwise justified here.

4. DISCUSSION POINTS

- Least privilege is iterative: start from required APIs, use Access Analyzer and CloudTrail, then tighten resources and conditions.
- Service roles and human roles need different trust policies and session durations; prevent confused-deputy cross-account access.
- IAM authorizes AWS resources, not document-level semantics. RAG ACL filters still require application identity and source permissions.

AWS CloudTrail

Authoritative AWS API audit evidence centralized outside the workload account.

1. ROLE

Records AWS control-plane activity and supported data events so investigators can determine who changed infrastructure, models, policies, objects, and key configurations.

2. TECHNOLOGY RATIONALE

The primitive is tamper-resistant API audit logging delivered to a centralized, access-restricted S3 trail, with organization coverage and optional high-value data events. Log-file validation and immutable retention support chain-of-custody; CloudTrail Lake can query selected audit events without operating a SIEM pipeline.

3. WHY THIS OVER ALTERNATIVES

Application logs explain business actions but do not authoritatively record AWS API calls. **CloudWatch Logs alone** can collect events but lacks the same organization-wide control-plane source semantics. **A third-party SIEM** is valuable downstream, not a substitute for generating native audit evidence.

4. DISCUSSION POINTS

- S3/Lambda data events are valuable but high-volume and costly; scope them to sensitive buckets and functions.
- Prevent the application team from modifying or deleting the organization trail; use a security account and Object Lock.
- CloudTrail often arrives with delay and is not a transaction log. Use it for audit/detection, not application correctness.

AWS Secrets Manager

Rotatable, audited retrieval of external credentials through IAM-authenticated workloads.

1. ROLE

Stores and rotates the OpenAI API credential and other non-AWS secrets, exposing them only to authorized runtime roles at execution time.

2. TECHNOLOGY RATIONALE

The primitive is encrypted, versioned secret retrieval with resource policies, rotation hooks, staging labels, and CloudTrail audit. Workloads authenticate to Secrets Manager with IAM and cache secret values briefly in memory; secret versions permit controlled rotation without embedding credentials in images, code, or environment files.

3. WHY THIS OVER ALTERNATIVES

Systems Manager Parameter Store is excellent for configuration and lower-cost secrets but has less purpose-built rotation workflow. **Environment variables** are convenient but leak through process, deployment, and diagnostic surfaces and are awkward to rotate. **HashiCorp Vault** is powerful and multi-cloud but adds a critical control plane to operate.

4. DISCUSSION POINTS

- Secrets Manager protects stored values, not accidental logging after retrieval; redact headers and provider responses.
- Rotation must account for providers that cannot overlap two active keys; stage, validate, and revoke with rollback.
- Cache carefully: per-request retrieval is slow and costly, while indefinite process caching delays revocation.

AWS WAF — cross-cutting policy

Central Layer-7 throttling and managed threat filtering before requests reach expensive inference.

1. ROLE

Applies centrally managed web protections across API endpoints, including managed threat rules, request-size controls, IP reputation, and rate-based limits.

2. TECHNOLOGY RATIONALE

The primitive is Layer-7 rule evaluation at the managed edge/API integration, with count mode, rule priorities, managed rule groups, custom conditions, and sampled logs. Organization-wide rule baselines can be deployed consistently while application teams add endpoint-specific limits. Rate rules absorb abusive bursts before they consume model tokens.

3. WHY THIS OVER ALTERNATIVES

Security groups operate at network layers and cannot inspect HTTP paths, headers, or payload patterns. **In-application middleware** is necessary for business validation but executes after capacity is consumed and can be bypassed by configuration mistakes.

CloudFront Functions are useful for lightweight edge logic, not a managed threat-rule engine.

4. DISCUSSION POINTS

- Run new managed rules in count mode before blocking; false positives can make the inspection UI unavailable at the worst moment.
- WAF body-inspection limits mean large prompts/documents need strict API contracts and application-layer validation.
- Do not claim WAF stops model prompt injection, authorization mistakes, or malicious retrieved documents—it does not.

CloudWatch Alarms

Independent metric evaluation that detects silent pipeline failure and pages from owned SLOs.

1. ROLE

Evaluates service, pipeline, data-freshness, and model-quality metrics against operational thresholds and routes actionable state changes.

2. TECHNOLOGY RATIONALE

The primitive is managed time-series threshold and anomaly evaluation with missing-data policy, metric math, composite alarms, and state transitions. Composite alarms combine symptoms—such as source freshness plus queue age—so operators receive incidents rather than every raw metric. High-severity alarms map to explicit runbooks and owners.

3. WHY THIS OVER ALTERNATIVES

Scheduled SQL checks can detect business anomalies but are delayed and add query failure modes. **Prometheus Alertmanager** is powerful for Kubernetes-centric systems but requires metrics infrastructure. **Email checks embedded in jobs** cannot detect that the job itself never ran.

4. DISCUSSION POINTS

- Missing data must be deliberate: “breaching” is correct for a freshness heartbeat, while “not breaching” may fit sparse errors.
- Static thresholds on seasonal traffic either flap or miss incidents; use anomaly detection only after enough baseline history.
- Alarm on user-impacting SLOs and causal signals, not every metric. Every page needs an owner and tested remediation.

Amazon SNS — operational notifications

Decoupled at-least-once fan-out from alarms to human and automated response channels.

1. ROLE

Routes alarm and security notifications to email, incident tooling, queues, and automated remediation subscribers without coupling producers to channels.

2. TECHNOLOGY RATIONALE

The primitive is at-least-once pub/sub fan-out with subscription protocols and filter policies. CloudWatch publishes alarm transitions; SNS distributes them to separate human and machine paths. Durable automation should receive through SQS, while email is a convenience channel rather than proof of handling.

3. WHY THIS OVER ALTERNATIVES

Direct Slack/email webhooks couple monitors to credentials and channel availability. **Amazon EventBridge** is stronger for complex event routing and SaaS targets but unnecessary for simple alarm fan-out. **PagerDuty-only integration** omits machine remediation and vendor-independent buffering.

4. DISCUSSION POINTS

- Duplicate notifications and state flapping are normal; deduplicate incidents by alarm ARN and state transition.
- Subscription confirmation, endpoint drift, and mailbox filtering can silently break delivery; test end to end.
- Do not send sensitive record content in notifications. Include a scoped console/runbook link and correlation ID.

Amazon QuickSight dashboards

Serverless governed BI for delayed cohort analysis, separate from operational monitoring.

1. ROLE

Provides governed analytical views of regulatory coverage, alert throughput, question hit-rate, evidence gaps, drift slices, and adoption trends for leaders and operators.

2. TECHNOLOGY RATIONALE

The primitive is serverless business intelligence over Athena/Redshift with SPICE in-memory acceleration, row-level security, scheduled refresh, and embedded dashboards. It fits aggregated, delayed analytical questions; operational pages stay in CloudWatch. SPICE isolates repeated dashboard reads from the raw lake and stabilizes query latency.

3. WHY THIS OVER ALTERNATIVES

CloudWatch dashboards are better for near-real-time operational telemetry but weak for rich cohort analysis and business joins.

Tableau/Power BI may be preferred enterprise standards but introduce licensing, gateway, and cross-cloud governance decisions.

Custom React charts maximize control at a high development and maintenance cost.

4. DISCUSSION POINTS

- SPICE capacity and refresh cadence can produce stale or expensive dashboards; display the data-as-of time.
- Row-level security must be tested against exported reports as well as interactive views.
- Avoid presenting hit-rate differences as causal business impact; show sample size, confidence, and cohort definition.

AWS Cost Explorer

Native AWS spend visibility for trend and anomaly diagnosis, complemented by CUR for unit cost.

1. ROLE

Provides account-level cost and usage analysis to identify service, environment, and trend drivers across ingestion, storage, search, ML, and external-model workloads.

2. TECHNOLOGY RATIONALE

The primitive is managed billing analytics over AWS cost-and-usage records with service, account, region, and tag dimensions. It supports trend analysis and anomaly investigation without maintaining a billing warehouse. For deeper unit economics, export the Cost and Usage Report and join it to application usage in Athena.

3. WHY THIS OVER ALTERNATIVES

CloudWatch metrics show resource activity but not authoritative billed cost, discounts, or amortization. **A custom cost warehouse** enables precise allocation but is unnecessary for first-line investigation. **OpenAI billing data** covers external tokens only and cannot explain AWS infrastructure cost.

4. DISCUSSION POINTS

- Cost Explorer has reporting delay and cannot drive hard real-time spend protection; use quotas and application token budgets.
- Shared serverless capacity is hard to allocate perfectly. Publish cost estimates and confidence, not false precision.
- Separate amortized, blended, and unblended views when discussing commitments; otherwise optimization conclusions can reverse.

Cost allocation tags

Governed billing dimensions for ownership, augmented by application metering for shared services.

1. ROLE

Attribute tagged resource cost to environment, product, owner, and—where technically possible—major workload such as ingestion, retrieval, or training.

2. TECHNOLOGY RATIONALE

The primitive is billing metadata propagated from AWS resource tags into cost reports after explicit activation. Tag policies and infrastructure-as-code enforce required keys; cost categories can group accounts and services where tags are unsupported. Application metering handles shared or untaggable serverless consumption.

3. WHY THIS OVER ALTERNATIVES

Separate AWS accounts only provide strong coarse isolation but cannot allocate shared platform cost by product or workload.

Resource names are not stable, governed billing dimensions.

Manual spreadsheets drift immediately and cannot reconcile to the bill.

4. DISCUSSION POINTS

- Tags are not retroactive in Cost Explorer and some resources inherit them inconsistently; activate early and audit coverage.
- Never put sensitive site or patient identifiers in tags—they propagate widely into billing systems.
- A tag cannot divide one shared OpenSearch collection or model call; combine tags with usage telemetry and an allocation rule.

Amazon S3 Object Lock — WORM policy

Policy-enforced immutability for evidence that must survive privileged deletion and later replay.

1. ROLE

Enforces retention and legal hold on source evidence, key audit trails, and selected model-decision records so authorized users cannot overwrite or delete them prematurely.

2. TECHNOLOGY RATIONALE

The primitive is version-level Write Once Read Many retention on versioned buckets. Governance mode permits tightly controlled override; compliance mode prevents deletion even by the root user until retention expires. Legal holds are independent of dates. The policy converts durability into immutability and supports defensible reconstruction of what informed a prediction.

3. WHY THIS OVER ALTERNATIVES

S3 versioning alone preserves previous versions but privileged principals can delete them. **A database audit table** can be altered by administrators and is expensive for large artifacts. **Glacier Vault Lock** offers archive controls but is less convenient for routinely queried evidence and modern S3 lifecycle workflows.

4. DISCUSSION POINTS

- Compliance mode is intentionally unforgiving; a mistaken retention period can lock petabytes and incur unavoidable cost.
- WORM storage protects bytes, not semantic correctness. Signed manifests, checksums, and lineage are still required.
- Replication must preserve retention and key access in the destination; test restore and legal-hold procedures, not only write protection.

One-line defense for every box

Use these as opening answers; expand into the detailed rationale and trade-offs when challenged.

Box	Why this technology	Box	Why this technology	Box	Why this technology
Regulatory feeds — FDA, EMA, ICH	Provenance-first polling for authoritative, heterogeneous sources.	OpenAI structured obligation extraction	Typed obligation extraction with exact source spans.	SageMaker Pipelines — retraining	Reproducible ML workflow with conditional quality gates.
Veeva Vault — documents and SOPs	Preserves controlled document versions, lifecycle, and ACLs.	Amazon Neptune — knowledge graph	Multi-hop applicability and provenance, not similarity.	SageMaker Model Registry — shadow, canary, production	Governed promotion through shadow, canary, production.
TrackWise — QMS records	Adds point-in-time quality signals with record history.	OpenAI embeddings — text-embedding-3-large	Versioned semantic vectors optimized for retrieval recall.	Amazon CloudWatch — hit-rate@10 and operations	Operational telemetry plus cohort-aware quality metrics.
SharePoint — LMS and files	Delta-aware file ingestion with source permissions intact.	OpenSearch Serverless — hybrid BM25 and vector index	Managed lexical plus vector retrieval with ACL filters.	AWS Key Management Service (KMS)	Central envelope encryption with auditable key policy.
Amazon EventBridge Scheduler	Serverless, independent, at-least-once scheduling.	SageMaker Feature Store — point-in-time features	Online latency plus leakage-safe historical feature joins.	AWS Identity and Access Management (IAM)	Short-lived identities and least privilege everywhere.
AWS Lambda — source connectors	Elastic compute for short I/O-bound connector runs.	Candidate generation — ANN, LLM, templates	Retrieval, generation, and templates maximize recall.	AWS CloudTrail	Authoritative AWS API audit outside workload accounts.
Amazon SQS with dead-letter queue	Durable backpressure with duplicate-safe delivery and DLQ.	LambdaMART ranker with MMR diversification	Fast learned ranking; MMR protects list-level coverage.	AWS Secrets Manager	Rotatable, audited retrieval of external credentials.
Amazon S3 raw lake with Object Lock	Immutable, versioned evidence for replay and audit.	Amazon ElastiCache — context-hash cache	Permission-aware reuse of expensive inference artifacts.	AWS WAF — cross-cutting policy	Layer-7 filtering before requests consume inference.
AWS Step Functions — diff pipeline	Inspectable orchestration with retries and human gates.	OpenAI grounded RAG — citations and NLI gate	Cited synthesis with entailment checks and abstention.	CloudWatch Alarms	Independent SLO evaluation that catches silent failure.
OpenAI materiality judge	Grounded semantic judgment for ambiguous document diffs.	Amazon API Gateway with AWS WAF	Managed API metering and protection for model capacity.	Amazon SNS — operational notifications	Decoupled fan-out to human and automated responders.
Amazon DynamoDB — change registry	Millisecond idempotency and document-version state.	Precompute worker — nightly and event-driven	Predictions ready before need; events keep them fresh.	Amazon QuickSight dashboards	Governed BI for delayed cohort and quality analysis.
Amazon SNS — impact alerts	Filtered fan-out after durable change registration.	Analyst Copilot UI	Task workflow exposes rationale and captures exposures.	AWS Cost Explorer	Native spend trends; CUR supplies detailed unit cost.
Amazon Textract — OCR and layout	Layout-aware OCR with coordinates and confidence.	Amazon Kinesis Data Firehose	Buffered at-least-once telemetry delivery to the lake.	Cost allocation tags	Billing ownership plus metering for shared services.
AWS Fargate — section-aware chunking	Flexible containers for long, variable parsing jobs.	S3 Parquet event lake with AWS Glue catalog	Open columnar history for analytics and retraining.	Amazon S3 Object Lock — WORM policy	Policy-enforced immutability for replayable evidence.